

DOCUMENTO DE PROJETO

Terços

Um app para casais dividirem o tempo com equilíbrio, se entenderem melhor e reduzirem brigas — desenhado sobre evidência de psicologia de casal.

Flutter · MVC

Go · Monolito modular / Bounded Contexts

PostgreSQL

Offline-first

Baseado em evidência

Plano de produto + arquitetura + fundamentos científicos · 07/07/2026

"Terços" é um nome de trabalho — escolhido pela mecânica central (divisão 3/3 do tempo). O nome definitivo e o esquema de deep-link ficam a definir.

Sumário

1. **Visão de produto** — problema, público, proposta de valor e o princípio-guia
2. **Fundamentos de psicologia de casal** — construtos, fontes e níveis de confiança
3. **Mapa Feature → Princípio → Risco → Salvaguarda** — e os alertas de design
4. **Arquitetura — visão geral** — monolito modular + 2 celulares + log de eventos
5. **Backend Go — bounded contexts** — camadas, pastas, eventos, stack
6. **Frontend Flutter — MVC** — abordagem, pastas, telas × controllers
7. **Modelo de domínio, banco e API** — por contexto
8. **Sincronização, notificações, auth e privacidade/LGPD**
9. **Roadmap MVP → v1 → v2**
10. **Salvaguardas éticas e de segurança**
11. **Questões em aberto** — decisões pendentes de produto/técnica
12. **Pré-requisitos e próximos passos**
13. **Referências**

COMO ESTE DOCUMENTO FOI PRODUZIDO

A fase de pesquisa rodou **20 subagentes em 2 workflows**: uma pesquisa multi-ângulo de psicologia de casal com **verificação adversarial** das afirmações de maior risco, e um design de arquitetura reconciliado a partir de 4 perspectivas independentes (backend, frontend, dados/API, sincronização). As decisões abaixo refletem essa pesquisa.

1. Visão de produto

Casais que convivem bem não são os que nunca discordam — são os que mantêm muito mais conexão positiva do que atrito, preservam a autonomia um do outro e reparam rápido os desentendimentos. **Terços** transforma esses hábitos em rotina prática.

1.1 Problema e público

O público-alvo são **casais que moram/convivem juntos** e querem: (a) equilibrar tempo individual e tempo a dois sem culpa; (b) decidir com justiça o que fazem juntos; (c) trocar feedback sem virar briga; e (d) perceber padrões de fricção antes que virem ressentimento. Hoje isso é feito "no improviso" e costuma escorregar para cobrança, placar mental de erros e desgaste.

1.2 As quatro mecânicas

- **Sistema de terços (3/3)**: a janela do dia dividida em três terços — *2 individuais* (um por parceiro) + *1 junto* — com registro de **início e fim** de cada terço (timestamps persistidos para auditoria e métricas).
- **Planejamento do terço junto**: catálogo de atividades, **rodízio diário** de quem escolhe e **1 veto por semana** por parceiro (reenquadrado como pausa/pedido, não como poder de vencer).
- **Feedback estruturado** — templates guiados que fazem *soft startup* e constroem também o lado de *quem recebe*.
- **Registro de fricção** — a antiga "camada de faltas", **reformulada**: notas privadas focadas em necessidade/impacto, revisão semanal calma e ciclo de reparo (ver §1.4 e §3).

1.3 Princípio-guia do produto: de déficit para superávit

DECISÃO CENTRAL DE PRODUTO

A métrica-mãe do app é a **razão apreciação : atrito** (positivo sobre negativo), **não** a contagem de erros. O app dá **ao menos o mesmo destaque** ao stream de apreço/gratidão que ao registro de fricção. Um app que notifica queixa mais do que apreço empurra o próprio casal para o negativo.

Isso tem lastro: o viés de negatividade ("o ruim é mais forte que o bom") é robusto (Baumeister et al., 2001) e casais estáveis mantêm muito mais interações positivas que negativas (Gottman, "razão mágica"). Usamos a **direção** do efeito — nunca um número cravado como meta (ver a caixa de calibração em §2).

1.4 O que Terços é — e o que deliberadamente NÃO é

É

- Um treinador de **hábitos baseados em evidência** (autonomia, apreço, reparo, decisão justa).
- Um espelho **compartilhado** e reversível dos padrões do casal.
- Privado por padrão no que é íntimo; transparente nas regras.

NÃO É

- Um **placar de culpas** nem um dossiê de erros do parceiro.
- Um oráculo que dá "% de chance de divórcio" ou nota ao relacionamento.
- Uma ferramenta de **vigilância** ou controle de um sobre o outro.

2. Fundamentos de psicologia de casal

Cada mecânica do app se apoia em construtos com respaldo na literatura. Abaixo, o que a ciência diz, o nível de confiança e as fontes — com uma calibração honesta sobre o que *não* podemos afirmar.

CALIBRAÇÃO HONESTA (REGRA DE OURO DO PRODUTO)

Os **construtos** e as **direções** de Gottman e afins são sólidos; os **números preditivos famosos não são**. "90% de acerto no divórcio", "96% pelos 3 primeiros minutos", "5:1", "86% vs 33%" vêm de modelos ajustados *post-hoc* em amostras pequenas, sem validação cruzada — desabam fora da amostra (Heyman & Slep, 2001; falha de replicação em Kim, Capaldi & Crosby, 2007). **O app se ancora nas direções** ("reduzir desprezo", "aumentar apreço", "apoiar autonomia") com linguagem de correlato — e **nunca** exibe score de risco ou "chance de divórcio" (seria falso, iatrogênico e viraria munição para desprezo).

2.1 Gottman / Quatro Cavaleiros construto: alta números preditivos: refutados

Crítica, **desprezo**, defensividade e *stonewalling* (retraimento) formam um mapa clínico útil de comunicação corrosiva. O **desprezo** (sarcasmo, desdém, posição de superioridade) é o marcador isolado mais robusto de sofrimento relacional. Cada um tem antídoto acionável: crítica → *soft startup*; defensividade → assumir parte da responsabilidade; desprezo → cultura de apreço; *stonewalling* → auto-acalmar-se diante do *flooding* fisiológico (o elo fisiológico é o mais bem ancorado — Levenson & Gottman, 1985).

Fontes: Gottman Institute (parte interessada — descrever, não como prova); Levenson & Gottman (1985, JPSP 49:85-94); Heyman & Smith Slep (2001, PMC1622921); Kim, Capaldi & Crosby (2007).

2.2 Razão 5:1 e viés de negatividade direção: alta número exato: contestado

Casais estáveis mantêm muito mais interações positivas que negativas; infelizes ficam perto de 1:1 ou menos. Conecta-se ao viés de negatividade robusto (o negativo pesa 2–5× mais). **Atenção**: o 5:1 de Gottman não se confunde com a "positivity ratio" de Losada (2,9:1), que foi **retratada** em 2013.

Fontes: Baumeister, Bratslavsky, Finkenauer & Vohs (2001); Gottman Institute; Brown, Sokal & Friedman (2013, retratação de Losada).

2.3 Autonomia / SDT / Diferenciação — a base dos terços autonomia/volição: alta

proporção 2/3-1/3: média/baixa

Autonomia e pertencimento **não são soma-zero**: apoiar a autonomia do parceiro melhora satisfação e ajuste diádico de ambos (SDT — Deci & Ryan). O tempo individual só beneficia quando é **volitivo** (escolhido, não fuga) — e o *solitude* voluntário de um parceiro beneficia o outro (estudo diádico). Preservar um *self* diferenciado (Bowen) prediz maior satisfação; fusão/*enmeshment* prediz conflito. O tempo junto sustenta a relação sobretudo quando é **auto-expansivo** (Aron: atividades novas/estimulantes), não co-presença passiva.

IMPLICAÇÃO PARA O "2/3 INDIVIDUAL + 1/3 JUNTO"

É um **default defensável**, não uma lei. Não existe proporção ideal única validada; 1/3 das horas acordado já supera o piso de conexão de Gottman (~6h/semana). O que torna o tempo individual saudável não é o número — é **volição** (foi desejado?), **simetria** (ambos querem?) e **qualidade do tempo junto**. Por isso a proporção é **calibrável** e o app mede volição/simetria, não só horas.

Fontes: SDT (Deci & Ryan); Weinstein et al. (PMC9132342); estudo diádico de *solitude* (PMC10736816); Bowen Family Systems; Aron et al. (*self-expansion*); Rusbult (modelo de investimento).

2.4 NVC / Feedback construtivo NVC/I-statements: média gratidão/apreço: alta

A CNV/NVC de Rosenberg estrutura a fala em Observação → Sentimento → Necessidade → Pedido — um **andaime de treino**, não fórmula mágica. Distinguir reclamação (comportamento) de crítica (caráter) é central. **Receber** feedback é habilidade distinta de dá-lo (Stone & Heen: gatilhos de verdade/relação/identidade). Gratidão/apreço é dos moduladores mais robustos, e o "ingrediente ativo" é o **outro-elogio** ("quando você fez X, mostrou que é Y"), dependente da responsividade percebida.

2.5 Comunal vs. Troca — o coração do reenquadramento das "faltas" alta

Relações íntimas saudáveis operam por normas **comunais** (dar por cuidado, com base na necessidade do outro), não por troca contabilizada. Rastrear *inputs* individuais é a assinatura do modo troca; o **próprio ato de manter placar** é marcador de deterioração e **causa** queda de satisfação. Nuance decisiva: casais comunais **monitoram necessidades, não a planilha de contribuições**. E o que prediz conflito é a **percepção** de injustiça, não o 50/50 aritmético.

Fontes: Clark & Mills (1979; 1993); Clark, Mills & Powell ("keeping track of needs"); Park & Impett et al. (pairfam, ~7.000 casais, 13 anos — "When Love Becomes a Ledger"); Walster, Walster & Berscheid (1978).

2.6 Gamificação saudável sobrejustificação: alta apps guiados (RCT): alta

Intervenções digitais para casais **funcionam quando estruturadas/guiadas** — o padrão-ouro é o OurRelationship (baseado em IBCT), com RCT de efeitos grandes, eficaz até com "coach" automatizado. Mas recompensar um gesto já prazeroso **corrói** a motivação intrínseca (sobrejustificação — Deci, 1971; Lepper). *Streaks* funcionam por aversão à perda, porém *backfire* (streak-anxiety). **Leaderboard/competição entre parceiros é antipadrão claro.**

REGRA DE GAMIFICAÇÃO

Nunca gamificar o afeto em si. Se houver gamificação, recompensar levemente o **processo** (aparecer, propor), com *streak do casal* (nunca individual) e "streak freeze" — jamais atribuir a quebra a uma pessoa (viraria arma de culpa).

Fontes: Doss et al. — OurRelationship RCT (PMC5786273); BMC Psychology (2025, s40359-025-03444-y); Deci (1971)/Lepper; SDT em mHealth (PMC8391751).

3. Mapa: Feature → Princípio → Risco → Salvaguarda

A tabela abaixo é a ponte entre a psicologia e o produto. Toda feature carrega um risco de virar o oposto do que promete; cada uma tem uma salvaguarda de design correspondente.

Feature	Princípio	Risco	Salvaguarda de design
Sistema de terços (2/3 individual + 1/3 junto)	SDT (autonomia/volição); Bowen (diferenciação); Aron (auto-expansão); Rusbult (investimento)	Proporção rígida patologiza casais saudáveis; assimetria oculta (um usa o individual para evitar, o outro se sente abandonado); subinvestir na conexão mina o <i>commitment</i> .	2/3-1/3 é default calibrável , não meta fixa. Medir volição ("recarregar ou fuga?") e simetria . Empurrar o terço junto para atividades novas/auto-expansivas . Norte = interdependência (self preservado + conexão), nem fusão nem afastamento.
Tracking de início/fim dos terços	SDT (volição > horas); equidade (percepção, não relógio)	Contabilizar horas transforma autonomia em obrigação e vira vigilância/ <i>scorekeeping</i> de tempo; média agregada esconde falta de consentimento.	Acoplar cada bloco a um check leve de autonomia , não a um cronômetro competitivo. Nunca comparar horas lado a lado . Faltar/adiar um dia é neutro , jamais "streak quebrado".
Rodízio de escolha + veto semanal	Justiça procedural (voz/controle/neutralidade); SDT; limite × controle coercitivo	O veto é a peça de maior risco : sem trilhos vira controle coercitivo, <i>stonewalling</i> institucionalizado, ou "banco de vetos" como ameaça; rodízio 50/50 rígido pode ser percebido como injusto.	Rodízio mantém-se (bom dispositivo de justiça). Veto SEMPRE com (a) necessidade própria enquadrada, (b) contraproposta, (c) loop de consentimento . Cooldown + 1/semana + sem acúmulo . Time-out estruturado (~20-30 min, retorno marcado) quando gera atrito.
Camada de feedback	NVC/OFNR; reclamação × crítica; <i>soft startup</i> ; Stone & Heen (receptor); Algoe (gratidão)	Campo livre vira canal de crítica/desprezo ; feedback só-texto assíncrono mata o reparo (sem tom); disparo em <i>flooding</i> = <i>harsh startup</i> garantido.	Composar guiado em 4 campos com lint que bloqueia "sempre/nunca" e adjetivos de caráter. Campo "entendo que você..." antes do pedido. Guard de timing (adiar 20 min, empurrar para presencial). Construir o lado receptor (check dos 3 gatilhos + acuse-recibo + botão de reparo).
Camada de "faltas" → registro de fricção	Comunal vs. troca; desprezo; viés de negatividade; SDT	Contador visível de erros do parceiro = <i>exchange ledger</i> literal — opera a variável que causa queda de satisfação e institucionaliza o "catalogar defeitos" que alimenta o desprezo; arquivo permanente vira "provas" armáveis.	Pivô central (adotado) : rastrear necessidades e impacto , nunca <i>inputs</i> /culpas. Reescrever como "diário de necessidades não-atendidas" em 1ª pessoa. Privado por padrão + cool-down ; compartilhar é opt-in. Fluxo de reparo nativo (marcar "reparada", decaimento). Sem contagem comparativa entre parceiros.
Métricas / auditoria	Razão 5:1 (direção); comunal vs. troca; SDT; OurRelationship (medir desfecho)	Placar de "quem errou mais" = <i>scorekeeping</i> ; dashboards de "monitorar o parceiro" viram vigilância; score de "risco de divórcio" = falso e iatrogênico ; otimizar por engajamento piora o bem-estar.	Métrica-headline = razão apreciação:atrito + taxa de reparo, como métricas compartilhadas ("nós vs. o problema"), sem leaderboard. Zero score determinístico . Detector de NSO : se positivos caem + atrito sobe + reparos são rejeitados, o app troca de modo (de "registrar problemas" para "reconstruir amizade") e sugere ajuda profissional.

3.1 Alertas de design (leitura obrigatória)

▲ O PLACAR DE FALTAS É O MODO DE FALHA Nº 1 DO PRODUTO

Um histórico cumulativo de erros do parceiro institucionaliza o "catalogar defeitos" — exatamente o solo do desprezo — e a evidência longitudinal mais forte (Park & Impett, ~7.000 casais / 13 anos) é direta: **manter o placar causa queda de satisfação**. Por isso a feature foi **reformulada, não removida** — o impulso por trás dela é legítimo (perceber padrões reais, como o tempo individual interrompido). A saída é **inverter a orientação**:

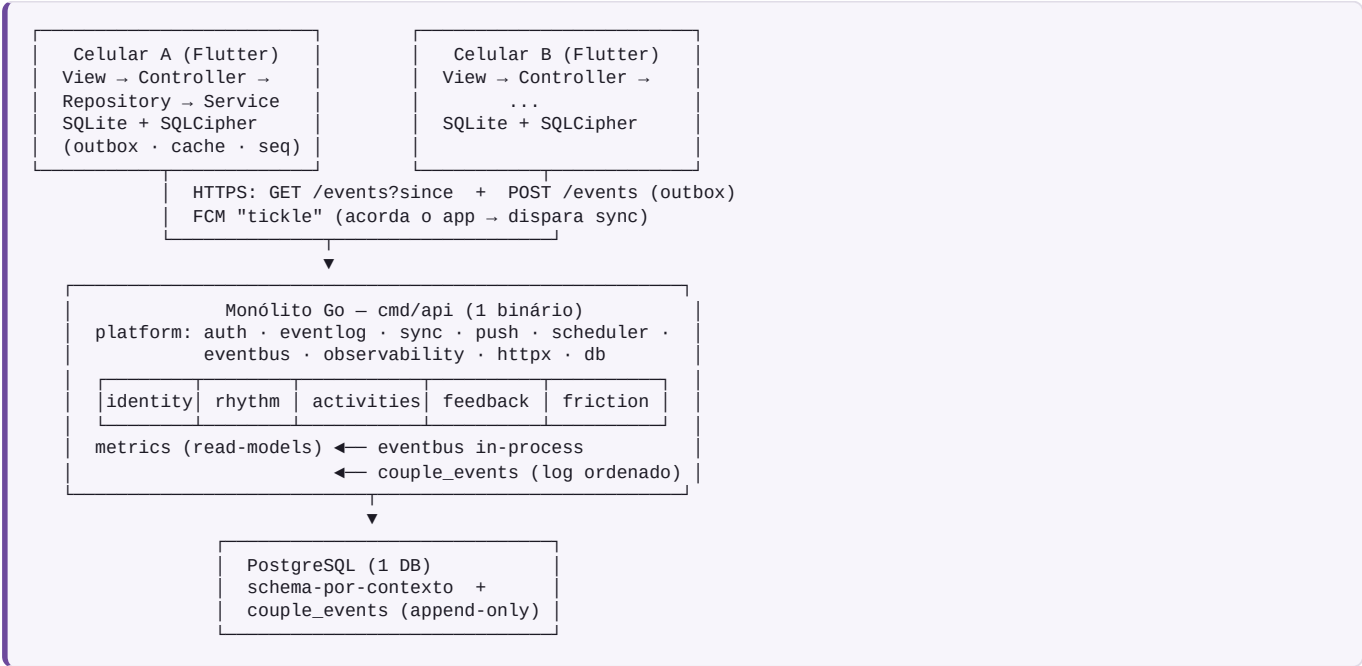
- **De inputs/culpas** → **para necessidades e impacto**. "Ele falhou X vezes" vira "minha necessidade de tempo individual ficou não-atendida quando Y, e o impacto foi Z".
- **De episódio** → **para padrão**. Nada de flag em tempo real no calor do conflito; entradas privadas + **revisão semanal calma** que agrega temas ("interrupção apareceu 4x").
- **De veredito** → **para pedido + reparo**. Toda entrada termina em necessidade + pedido e tem estado "reparada" (com decaimento). Resolver e soltar, não acumular provas.
- **De vocabulário de acusação** → **para vocabulário de necessidade**. Reconsiderar os próprios nomes "falta"/"veto" (avaliações de caráter que convidam defensividade).

▲ ZERO DETERMINISMO · ZERO VIGILÂNCIA · TRIAGEM DE SEGURANÇA

Nada de "probabilidade de divórcio" ou nota do relacionamento (a acurácia prospectiva não existe — VPP ~21-29% fora da amostra — e vira profecia autorrealizável). Nada de comparação parceiro-vs-parceiro nem histórico imutável usado como dossiê; visibilidade só com consentimento **mútuo e simétrico**. Todo o framework pressupõe **simetria entre parceiros** e é inválido sob abuso/controle coercitivo — incluir salvaguardas e caminhos de saída/ajuda profissional (§10).

4. Arquitetura — visão geral

Monólito modular Go (**1 binário, 1 processo, 1 Postgres**), multi-tenant por **CoupleID**, com 2 dispositivos por casal. Cada celular roda Flutter em MVC. A espinha dorsal é um **log append-only por casal** que serve de canal de sync, fila de reconciliação offline, trilha de auditoria e fonte das métricas.



4.1 Vocabulário canônico (fonte única de nomes)

Bounded contexts (internal/): identity rhythm activities feedback friction metrics . **Cross-cutting** (internal/platform/): auth eventlog sync push scheduler eventbus observability config db httpx server . **Shared kernel**: internal/kernel .

Notificações **não** são bounded context (não têm invariante de negócio) — viram capacidade de platform (push + scheduler + feed in-app).

AGREGADOS POR CONTEXTO

Contexto	Agregados
identity	User, Couple, Membership, Invitation, Device
rhythm	DailyRhythm (raiz) → Third[3] · ThirdType {individual_a, individual_b, shared}
activities	ActivityCandidate, ChooserRotation, ActivityPlan, Veto
feedback	FeedbackEntry, FeedbackTemplate
friction	FrictionNote (privado por padrão, sem "culpado")
metrics	read-models WeeklyTimeSplit, VetoUsage, FrictionTrend, FeedbackSentiment

CATÁLOGO DE EVENTOS DE DOMÍNIO (GRAVADOS EM COUPLE_EVENTS)

Contexto	Eventos
identity	UserRegistered · CoupleFormed · PartnerJoined · DeviceRegistered
rhythm	RhythmCreated · ThirdStarted · ThirdEnded · ThirdSkipped · RhythmCompleted
activities	ActivityProposed · VetoUsed · ChoiceConfirmed · TurnAdvanced
feedback	FeedbackShared · FeedbackRead
friction	FrictionNoted · FrictionResolved

5. Backend Go — monolito modular / bounded contexts

Regra de dependência: `port → app → domain ← infra`. O `domain` é Go puro (só importa `kernel` + `stdlib`). Nenhum `internal/<A>` importa `domain / infra` de `internal/<B>` — o cruzamento só acontece por **interface consumer-defined** (síncrono, fiada no composition root) ou por **evento** (assíncrono in-process).

5.1 Árvore de pastas

```
couples-backend/
├── cmd/api/main.go           # composition root: config, pool, eventbus, wiring, http server
├── internal/
│   ├── kernel/              # SHARED KERNEL (tipos de domínio puros)
│   │   ├── ids.go           # UserID, CoupleID, MembershipID, DeviceID (UUIDv7), EventID
│   │   ├── time.go          # Clock, LocalDate, TZ, WeekKey
│   │   ├── event.go         # DomainEvent, PartnerSlot
│   │   └── errors.go        # ErrConflict, ErrNotFound, ErrForbidden
│   ├── identity/ { module.go, domain/, app/, infra/, port/ }
│   ├── rhythm/ { module.go, domain/, app/, infra/, port/ }
│   ├── activities/ { module.go, domain/, app/, infra/, port/ }
│   ├── feedback/ { module.go, domain/, app/, infra/, port/ }
│   ├── friction/ { module.go, domain/, app/, infra/, port/ }
│   └── metrics/ { module.go, app/ (projectors+queries), infra/, port/ }
├── platform/
│   ├── config/ # Load() -> Config (12-factor/env)
│   ├── db/ # pgxpool, UnitOfWork/Tx, health
│   ├── eventlog/ # Append() transaccional (atribui server_seq), dedup por client_event_id
│   ├── eventbus/ # pub/sub in-process (canais Go) alimentado pelo log
│   ├── sync/ # Delta(reader, since) filtrado por visibility; Hub SSE (v2)
│   ├── push/ # cliente FCM (tickle) + device tokens
│   ├── scheduler/ # scheduled_notifications + Worker (SELECT ... FOR UPDATE SKIP LOCKED)
│   ├── auth/ # JWT issue/verify, refresh rotativo, middleware coupleID, argon2id
│   ├── httpx/ # ServeMux, middlewares, erro->problem+json (RFC7807)
│   ├── observability/ # slog, OpenTelemetry, Prometheus
│   └── server/ # http.Server + graceful shutdown
├── pkg/ { idgen/, problemjson/, paginate/ } # utilitário genérico, não-domínio
├── migrations/ # goose, um dir por schema/contexto
├── sqlc.yaml
├── go.mod
└── Taskfile.yml
```

Camadas por contexto: **domain/** (agregados, VOs, eventos, interfaces de repositório — não importa `infra/http/pgx`); **app/** (use cases, limite transaccional, interfaces consumer-defined); **infra/** (`pgx/sqlc`, mapeia row ↔ agregado); **port/** (handlers HTTP + subscribers do eventbus).

5.2 Comunicação por eventos (o backbone)

Append transaccional unificado: todo use case que muda estado grava, **na mesma transação**, (a) o snapshot do agregado e (b) o append em `couple_events`. É o *transactional outbox* e a auditoria ao mesmo tempo — nunca há "estado mudou mas evento sumiu". Um *relay* (goroutine) lê o log por `server_seq` e publica no `eventbus` in-process; consumidores: `metrics` (projeções), `push / scheduler` (notificações). At-least-once + handlers idempotentes por `server_seq`.

```
// platform/eventlog
type Event struct {
    ClientEventID uuid.UUID          // gerado no device -> idempotência/dedup
    CoupleID      kernel.CoupleID
    ActorMember   kernel.MembershipID
    DeviceID      kernel.DeviceID
    Type          string              // "ThirdStarted", "VetoUsed", ...
    Payload       []byte              // jsonb tipado + Version
    ClientTS      time.Time          // relógio do device (medição offline)
    ServerSeq     int64              // atribuído pelo servidor, MONOTÔNICO POR CASAL
    ServerTS      time.Time
    Visibility    Visibility          // shared | private_author
}
func Append(ctx, tx, e Event) (Event, error) // dedup por ClientEventID; PK (couple_id, server_seq)
```

Síncrono cross-context: interface declarada em `A/app`, implementada pelo `Module` de B, fiada no composition root. Ex.:

`activities/app` declara `type RhythmQuery interface { SharedThird(ctx, CoupleID, LocalDate) (ThirdRef, error) }`.

5.3 Stack

Aspecto	Escolha
Roteador	<code>net/http</code> ServeMux 1.22+ (Go presente no host); chi só se rotas/middlewares por grupo crescerem
Postgres	<code>pgx/v5</code> + <code>sqlc</code> (type-safe, sem ORM); um pacote sqlc por contexto
Migrations	<code>goose</code> (embed.FS, SQL+Go), um dir por schema
Auth	JWT <code>go lang-jwt/v5</code> , assimétrico RS256/EdDSA, access ~15min; refresh rotativo, hashado, por device, com detecção de reuso; senha <code>argon2id</code>
Config / Logs	12-factor via env; <code>log/slog</code> (JSON prod), OpenTelemetry, Prometheus
IDs	<code>UUIDv7</code> (ordenável) no kernel; <code>server_seq</code> BIGSERIAL por casal
Isolamento no banco	schema-por-contexto num único Postgres, sem FK cruzando schema; referência cross-context só por ID. Toda tabela tem <code>couple_id</code> e toda query filtra por ele a partir do token (à prova de IDOR). RLS do Postgres é evolução (v2)

6. Frontend Flutter — MVC

6.1 Abordagem

MVC via Provider + ChangeNotifier (Controller = `ChangeNotifier` com **injeção por construtor**; Model = models imutáveis `freezed` + Repositories + Services; View = widgets puros com `switch` exaustivo sobre `ViewState<T>`). Escolhido sobre GetX (mágica/acoplamento global) e Stacked (é MVVM) por manter fronteiras explícitas e testabilidade coerentes com o DDD-lite do backend.

- **Regra de dependência:** View → Controller → Repository → Service. A View nunca toca Repository/Service; o Controller nunca conhece `dio` /DB/ `BuildContext`.
- **Navegação:** `go_router` (redirect guard: não-auth → welcome; auth não-pareado → pareamento; pareado → shell; deep links de notificação).
- **DI:** `get_it` + `injectable` (services/repos `@lazySingleton`; controllers `@injectable` por rota).
- **API:** `dio`, **um client por bounded context**, com interceptors (auth+refresh, retry, log, error → `Failure`). DTOs separados das entidades via mappers. Repositories retornam `Result<T, Failure>`.
- **Offline-first:** `drift` (SQLite) + `SQLCipher` (`outbox`, `event_log_cache`, `materialized_state`, `last_seen_seq`); `flutter_secure_storage` para tokens e a chave do `SQLCipher`.
- **Sync:** `SyncEngine` (único a escrever `materialized_state`) drena a outbox (`POST /events`), faz pull incremental (`GET /events?since=seq`) e reconcilia; `firebase_messaging` (data message = *tickle*) dispara sync.

6.2 Árvore de pastas

```
lib/  
  main.dart · app.dart                # bootstrap + MultiProvider raiz (app-scoped)  
  core/  
    theme/ routing/ di/ constants/  
    network/ # dio_client, interceptors, error_mapper  
    db/      # drift + SQLCipher; daos: outbox, cache, materialized_state, last_seen_seq  
    sync/    # sync_engine, outbox_flusher (backoff), sse_client (v2)  
    push/    # fcm handler (tickle -> sync)  
    auth/    # secure_storage, token_store  
    time/    # clock (offset vs servidor), result, view_state  
  models/   # POR BOUNDED CONTEXT: entity(freezed)+dto+mapper  
    couple/ thirds/ activity/ feedback/ friction/ metrics/ common/  
  repositories/ # auth · couple · thirds · activity · feedback · friction · metrics · settings  
  services/    # api/ auth/ notifications/ connectivity/ local/  
  controllers/ # onboarding/ today/ rhythm/ planning/ feedback/ friction/ metrics/ settings/ sync/  
  views/       # onboarding/ today/ rhythm/ planning/ feedback/ friction/ metrics/ settings/ shell/ shared/
```

6.3 Telas × Controllers × Models × Repositórios

Tela(s)	Controller(s)	Models	Repos
Welcome / SignIn / CreateCouple / JoinCouple / PairingWait	<code>OnboardingController</code> , <code>PairingController</code>	<code>Partner</code> , <code>Couple</code> , <code>InviteCode</code> , <code>AuthSession</code>	<code>AuthRepository</code> , <code>CoupleRepository</code>
Today (3 terços, timer start/stop)	<code>TodayController</code> + <code>ThirdsTimerController</code>	<code>ThirdsDay</code> , <code>ThirdSession</code> , <code>ThirdType</code> , <code>TimerState</code>	<code>ThirdsRepository</code>
RhythmHistory (timeline, split bar)	<code>RhythmHistoryController</code>	<code>ThirdsDay</code> , <code>ThirdSession</code>	<code>ThirdsRepository</code>
Planning (candidatas, "de quem é a vez", veto, escolha)	<code>ActivityPlanningController</code>	<code>Activity</code> , <code>ActivityRotation</code> , <code>Veto</code> , <code>DailyChoice</code>	<code>ActivityRepository</code>
FeedbackInbox / Compose / Detail	<code>FeedbackController</code> , <code>ComposeFeedbackController</code>	<code>Feedback</code> , <code>FeedbackTemplate</code> , <code>Sentiment</code>	<code>FeedbackRepository</code>

Tela(s)	Controller(s)	Models	Repos
FrictionLog / AddNote	FrictionController	FrictionNote , FrictionCategory , Visibility	FrictionRepository
MetricsDashboard (fl_chart)	MetricsController	MetricSnapshot , TimeSplitStat , VetoUsageStat , SentimentTrend , FrictionTrend	MetricsRepository
Settings / ThirdsConfig / Notifications	SettingsController	CoupleSettings , ThirdsConfig , NotificationPrefs	SettingsRepository

Timer offline-first: o device persiste `started_at` (wall clock); o cronômetro é `clock.now() - started_at` (derivado do timestamp, não contador em memória) → sobrevive a app kill/reopen e não sofre drift; `Timer.periodic(1s)` só redesenha.

7. Modelo de domínio, banco e API (por contexto)

Convenções: JSON, `Authorization: Bearer`; **escopo do casal derivado do token, nunca do path/body**; **escrita de domínio via POST /events** (idempotente por `client_event_id`); reads via GET; respostas trazem `version` / `updated_at` para concorrência otimista. Base path `/v1`.

7.1 identity

```
users(id PK, email UNIQUE, display_name, auth_provider, created_at)
user_devices(id PK, user_id FK, push_token, platform, last_seen_at)
couples(id PK, status, timezone, week_starts_on SMALLINT, default_rhythm_config JSONB, created_at, dissolved_at)
memberships(id PK, couple_id FK, user_id FK, partner_slot CHAR(1) CHECK(A|B), status, joined_at, left_at,
             UNIQUE(couple_id, partner_slot), UNIQUE(couple_id, user_id)) -- máx. 2 ativos
pairing_invites(token PK, couple_id FK, invited_by FK, status, expires_at, used_at, created_at)
```

Método	Path	Resumo
POST	/auth/register · /auth/login	cria usuário / login
POST	/auth/refresh	rotaciona refresh (detecção de reuso)
POST	/devices	registra device + push token
POST	/couples	cria casal (<code>pending_pairing</code>) → <code>CoupleFormed</code>
POST	/pair/invite	gera token opaco + deep link <code>app://pair?token=...</code>
POST	/pair/accept	aceita convite (atômico <code>WHERE used_at IS NULL</code>) → <code>PartnerJoined</code>
GET	/couples/me	casal + memberships + config
PATCH	/couples/me/config	timezone, week_starts_on, config padrão de terços

7.2 rhythm (núcleo)

`DailyRhythm{..., Status(planned|in_progress|completed|abandoned), Mode(sequential|parallel), [3]Third}, unique(couple_id, local_date)`. O **timestamp persistido é a fonte da verdade**: `StartedAt` / `EndedAt` são fatos imutáveis enfileirados na outbox — não há conflito a resolver. *Rejeitado event-sourcing pleno* (overhead para N=2): snapshot mutável + log append-only.

```
daily_rhythms(id PK, couple_id FK, local_date DATE, status, mode, config JSONB, created_at,
              UNIQUE(couple_id, local_date))
thirds(id PK, daily_rhythm_id FK, couple_id FK, type, owner_membership_id FK NULL,
        planned_start_at, planned_end_at, planned_duration_sec, status,
        started_at TIMESTAMPTZ NULL, ended_at TIMESTAMPTZ NULL,
        actual_duration_sec INT GENERATED, updated_at, version INT)
```

Método	Path	Resumo
POST	/rhythm/days	cria/garante o DailyRhythm do dia
GET	/rhythm/today · /rhythm/days/{date}	terços + status
GET	/rhythm/days/{date}/audit	log (couple_events filtrado por rhythm) do dia
PATCH	/rhythm/thirds/{id}	ajusta plano (flexibilidade)

Mutações via `POST /events`: `ThirdStarted`, `ThirdEnded`, `ThirdSkipped`, `RhythmCompleted` — idempotentes + concorrência otimista (`version`) no terço `shared`.

7.3 activities

Rodízio por uso: o ponteiro só avança no `ChoiceConfirmed` (dia pulado não consome a vez); `WhoseTurn(date)` é derivado do log ordenado (converge nos 2 devices sem sincronizar célula mutável). **Veto = 1/semana por parceiro, reset SEM cron**:

`UNIQUE(couple_id, by_membership_id, week_key)` impõe o limite e o virar da semana zero sozinho. O veto **obriga o mesmo escolhedor a re-escolher** (não passa a vez); candidato vetado entra em `rejected_candidate_ids`.

```

activity_candidates(id PK, couple_id FK, title, description, category, estimated_duration_sec,
    created_by_membership_id FK, is_active, created_at, archived_at)
chooser_rotations(couple_id PK/FK, last_chooser_membership_id FK NULL, last_chooser_date DATE NULL,
    anchor_slot CHAR(1), updated_at)
activity_plans(id PK, couple_id FK, daily_rhythm_id FK, shared_third_id FK, plan_date DATE,
    chooser_membership_id FK, chosen_candidate_id FK NULL, status,
    rejected_candidate_ids UUID[], confirmed_at, created_at, version INT,
    UNIQUE(couple_id, shared_third_id))
vetoes(id PK, couple_id FK, activity_plan_id FK, by_membership_id FK, target_candidate_id FK,
    week_key TEXT, used_at, UNIQUE(couple_id, by_membership_id, week_key))

```

Método	Path	Resumo
GET/POST/PATCH	/activities/candidates/{id}	catálogo (arquiva por flag)
GET	/activities/rotation/today	de quem é a vez (derivado)
GET	/activities/vetoes/allowance	saldo de veto na semana (por membership)

Mutações via `POST /events`: `ActivityProposed`, `VetoUsed`, `ChoiceConfirmed` (avança rodízio → `TurnAdvanced`). O servidor valida quota e "de quem é a vez"; o client faz UI otimista e reverte se rejeitado.

7.4 feedback

Bidirecional; **draft é privado do autor** (`visibility=private_author`) e só se torna visível no `FeedbackShared`. Template guiado padroniza tom e captura `Sentiment`.

```

feedback_templates(id PK, code UNIQUE, name, prompts JSONB, tone, is_active)
feedback_entries(id PK, couple_id FK, author_membership_id FK, recipient_membership_id FK,
    template_id FK, answers JSONB, sentiment SMALLINT, related_third_id FK NULL,
    related_friction_note_id FK NULL, status, created_at, shared_at)

```

7.5 friction — reformulado (segurança psicológica)

A REFORMULAÇÃO, NO BANCO

Sem campo "culpado" e sem contador por parceiro. A nota nasce **privada** (`visibility=private_author`); o `sync.Delta` filtra por leitor — nota privada nunca entra no delta do parceiro. As métricas agregam por categoria/impacto/resolução, **nunca** "quem errou mais". Ciclo nativo notar → (opcional compartilhar) → resolver.

```

friction_notes(id PK, couple_id FK, reporter_membership_id FK,
    category,          -- interrupcao | tom | expectativa | tarefa | tempo | outro
    impact,            -- low | medium | high (como afetou QUEM relatou)
    description,       -- guiada por impacto/necessidade (1ª pessoa)
    occurred_at, visibility, -- private_author (default) | shared_with_couple
    resolved BOOL, resolved_at, resolution_note, created_at)

```

Mutações via `POST /events`: `FrictionNoted`, `FrictionResolved` (só as `shared_with_couple` notificam o parceiro).

7.6 metrics (read-side, CQRS-lite)

Consome `couple_events` → projeções `rm_*` (idempotentes por `server_seq`; podem começar como *materialized view* no MVP). O dashboard lê só as tabelas `rm_*`. **Headline = razão apreciação:atrito + taxa de reparo**, como métricas compartilhadas.

```

rm_weekly_time_split(couple_id, week_key, individual_a_sec, individual_b_sec, shared_sec,
    thirds_completed, thirds_planned, updated_at, PK(couple_id, week_key))
rm_weekly_veto_usage(couple_id, week_key, membership_id, vetoes_used, PK(couple_id, week_key, membership_id))
rm_friction_trend(couple_id, period_key, category, count, impact_high, resolved_count,
    avg_resolution_hours, PK(couple_id, period_key, category))
rm_feedback_sentiment(couple_id, week_key, positive, neutral, negative, top_template, PK(couple_id, week_key))

```

7.7 O log e o sync (schema **platform**)

```

couple_events(couple_id, server_seq BIGSERIAL, client_event_id UUID, actor_membership_id, device_id,
              type, payload JSONB, client_ts, server_ts, visibility,
              PRIMARY KEY(couple_id, server_seq),
              UNIQUE(couple_id, client_event_id))          -- dedup/idempotência
scheduled_notifications(id PK, couple_id, membership_id, kind, fire_at TIMESTAMPTZ,
                       status, idempotency_key UNIQUE, created_at)
refresh_tokens(id PK, user_id, device_id, hash, expires_at, revoked)

```

Método	Path	Resumo
POST	/couples/me/events	flush do outbox (lote): idempotente: roteia p/ handler do contexto: retorna {client_event_id → server_seq rejected+reason}
GET	/couples/me/events?since={seq}	delta pull, filtrado por visibility do leitor ; ETag/304
GET	/couples/me/sync/stream	SSE de presença ao vivo (v2)

8. Sincronização, notificações, auth e privacidade/LGPD

8.1 Sincronização entre os 2 celulares

Servidor é a **fonte da verdade**; apps fazem UI otimista + reconciliação sobre o log.

- **MVP:** REST delta (`GET /events?since=seq`, ETag/304) + **FCM como "tickle"** (data message mínimo "há novidade, $seq \geq N$ " que acorda o app para sincronizar — FCM nunca carrega estado autoritativo). Foreground: poll adaptativo (~5–10s ativo, ~30–60s ocioso, backoff no 304).
- **v2:** SSE (não WebSocket — o fluxo dominante é server → client) para presença ao vivo no terço junto; hub in-process → Redis/NATS como backplane ao escalar.

8.2 Offline-first e conflitos

Conflito quase inexistente por partição de dono (single-writer): terço individual (dono = 1 parceiro), veto (orçamento próprio), rascunho/nota de fricção (privado). O **rodízio** é derivado do log (não sincroniza célula mutável). Só o **terço junto/escolha do dia** é célula compartilhada → **LWW por `server_seq`** (ordem total do servidor), com `version` /idempotência garantindo 1 sessão se ambos iniciarem. *Vector clock explicitamente rejeitado* (overkill para N=2 majoritariamente single-writer).

8.3 Notificações e scheduler

- **Imediatas (por evento):** consumidor do `eventbus` dispara FCM tickle ao ver `ThirdStarted`, `ChoiceConfirmed`, `VetoUsed`, `FeedbackShared`, `FrictionNoted` (se *shared*). Toda notificação também vira item no **feed in-app** (lido do log).
- **Agendadas (por tempo):** "sua vez de escolher hoje", lembrete de reset semanal de veto, lembrete de feedback — tabela durável `scheduled_notifications` + **worker goroutine** com `SELECT ... FOR UPDATE SKIP LOCKED` (sobrevive a restart, multi-instância safe, mantém 1 deployable). `fire_at` em UTC a partir de `couple.timezone`.

8.4 Auth / tenant

Pareamento: A cria casal → gera token opaco single-use com TTL (código curto + deep link com App/Universal Links); B aceita (`UPDATE ... WHERE used_at IS NULL`, atômico) → adiciona membership, trava >2 membros. **Tokens:** access JWT ~15min com claims `sub/couple/mid/slot/did`; refresh rotativo, hashado por device, com detecção de reuso. **Tenant:** `couple_id` vem do **token**, injetado no `context`; todo repositório filtra por ele e jamais confia em `couple_id` do body/param.

8.5 Privacidade e LGPD

CLASSIFICAÇÃO DE DADOS

- **Privado do autor:** rascunho de feedback, nota de fricção, reflexão.
- **Compartilhado no casal:** terços, escolha, vez, *fato* de veto, feedback enviado, métricas.
- **Só sistema:** tokens, hash de refresh, log cru.

CRIPTO EM CAMADAS

- TLS 1.2+ (cert pinning na v2); coluna cifrada (KMS) nos textos íntimos no servidor.
- SQLCipher no SQLite do device; tokens no Keychain/Keystore.
- **E2EE adiado** (conflita com métricas/sentimento server-side) → v2 para conteúdo `private_author`.

LGPD: consentimento explícito, exportação e apagamento (por conta, unilateral), minimização; log append-only apagável por **crypto-shredding** (apagar a chave de coluna torna o texto irrecoverável sem reescrever o log). Nada de texto íntimo em logs/métricas.

9. Roadmap MVP → v1 → v2

Fase	Entregas
MVP o loop essencial entre 2 celulares	<code>platform</code> (config, db, <code>eventlog</code> + <code>couple_events</code> , <code>eventbus</code> , <code>auth</code> , <code>httpx</code> , <code>push</code> <code>tickle</code> , <code>scheduler</code> base) · <code>identity</code> (pareamento por token opaco + JWT/refresh) · <code>rhythm</code> (3 terços, start/end com timestamps, timer offline-first) · <code>activities</code> (rodízio por uso derivado do log + veto 1/semana com contraproposta) · <code>feedback</code> (templates guiados, draft privado, share) · Sync REST delta + <code>POST /events</code> + FCM tickle · Privacidade base (TLS, coluna cifrada, SQLCipher) · Flutter : onboarding, Today+timer, Planning, Feedback, SyncEngine.
v1 insight, ritual e conformidade	<code>metrics</code> (projeções <code>rm_*</code> + dashboards <code>fl_chart</code> , headline <code>apreço:atrato</code>) · <code>friction</code> reformulado (visibility, ciclo notar → resolver, revisão semanal) · <code>scheduler</code> completo (sua-vez, reset de veto, lembrete de feedback) + feed in-app · endpoints de auditoria · LGPD (exportação/deleção unilateral, crypto-shredding) · observabilidade de qualidade (terços "pendurados", skew de relógio, backlog de outbox) · detector de NSO + contrapeso positivo (stream de <code>apreço</code>).
v2 tempo real e blindagem	SSE (presença ao vivo no terço junto) + backplane Redis/NATS ao escalar · RLS Postgres por <code>couple_id</code> · cert pinning · E2EE para conteúdo <code>private_author</code> · fluxo de dissolução do casal · golden tests.

O **backbone caro de retrofitar já entra no MVP**: `couple_events`, schema-por-contexto, idempotência e timestamps persistidos.

10. Salvaguardas éticas e de segurança

- **Simetria pressuposta.** Todo o framework de casal vale sob simetria entre parceiros e é **inválido sob abuso/controle coercitivo**. Rastreamento unilateral e secreto pode virar ferramenta de abuso → incluir triagem, limites e caminho de saída/ajuda profissional.
- **Zero determinismo.** Nunca exibir "% de chance de divórcio" ou nota do relacionamento — a acurácia prospectiva não existe (VPP ~21-29% fora da amostra) e vira profecia autorrealizável.
- **Contrapeso positivo de primeira classe.** O stream de apreço/gratidão tem **ao menos o mesmo destaque** que o registro de atrito — antídoto ao desprezo e ao viés de negatividade.
- **Sem vigilância.** Nada de comparação parceiro-vs-parceiro, screenshot/forward como "prova" ou histórico imutável usado como dossiê. Visibilidade só com consentimento mútuo e simétrico.
- **Marketing honesto.** O diferencial é "**hábitos baseados em evidência**", nunca "algoritmo que prevê seu divórcio". Citar construtos/direções com linguagem de correlato.
- **Detector de NSO → troca de modo.** Ao detectar *negative sentiment override* (positivos caindo + atrito subindo + reparos rejeitados), o app deixa de empurrar feedback duro e passa a reconstruir amizade (bids, apreço, love maps), sugerindo apoio profissional.

11. Questões em aberto

Decisões pendentes que afetam produto e/ou modelagem. Recomendações entre parênteses.

1. **Fuso / relação à distância:** parceiros podem estar em fusos diferentes? O "dia" e o reset do veto usam o fuso **do casal** (recomendado) ou por parceiro? Afeta `(couple_id, local_date)` e `WeekKey`.
2. **Semana do veto:** começa quando (segunda 00:00? domingo?) e em qual fuso?
3. **Rodízio:** confirmar "**por uso**" (recomendado, derivado do log) vs. "por calendário". O que acontece se ninguém confirmar por vários dias?
4. **Política do veto:** a janela é só **antes de confirmar** o plano, ou também depois de iniciado o terço? O veto conta contra quem vetou mesmo que o escolhedor não troque?
5. **Feedback:** o `Sentiment` é autoinformado pelo autor, avaliado pelo receptor ou derivado por NLP?
6. **Fricção (produto/segurança):** a nota é 100% invisível ao parceiro, o parceiro vê que existe, ou é `private_author` por padrão com opt-in de compartilhar (recomendado)?
7. **Auth do MVP:** magic-link por email, social (Google/Apple) ou senha? Afeta `users` / `user_devices`, refresh e onboarding.
8. **Push:** conta Firebase/APNs definida? Firebase é aceitável (privacidade/vendor)? MVP pode começar só com pull e adicionar push depois?
9. **Modo paralelo:** permitir mais de um terço `active` simultâneo (A e B ao mesmo tempo)? No `rm_weekly_time_split` tempos sobrepostos somam ou viram janela única?
10. **Janelas por dia:** um único `DailyRhythm` por casal/dia (recomendado) ou múltiplas janelas?
1. **E2EE:** é requisito ter servidor cego para algum conteúdo, ou coluna cifrada server-side atende à promessa no MVP (impacta métricas/sentimento)?
2. **LGPD / retenção:** crypto-shredding é aceitável como "apagamento" do `couple_events`, ou exige remoção física? Deleção de conta é unilateral e afeta o histórico do parceiro?
3. **Dissolução do casal:** ao desaparecer, o histórico/métricas — cada um leva cópia, congela ou apaga?
4. **Escala:** volume esperado de casais (para dimensionar a troca de polling+FCM por SSE e a introdução de backplane).
5. **Nome/branding e deep-link:** definir nome definitivo (hoje "Terços") e o esquema `app://pair`.

12. Pré-requisitos e próximos passos

ESTADO DO AMBIENTE LOCAL (VERIFICADO EM 07/07/2026)

`go` presente · `chromium` e `libreoffice` presentes (usados para gerar este PDF) · `flutter` / `dart` ausentes · `node`, `python3` presentes. Postgres/Firebase a provisionar.

1. **Ferramental Flutter:** instalar Flutter/Dart SDK (Android Studio/Xcode toolchains) — pré-requisito para o app.
2. **Repositórios:** criar `couples-backend` (Go) e `couples-app` (Flutter) com a estrutura das §5.1 e §6.2 — *o scaffold é o próximo passo natural após este plano.*
3. **Infra:** subir Postgres local (docker-compose), configurar `goose` e `sqlc`, criar os schemas por contexto.
4. **Push:** decidir Firebase/APNs (Questão 8) e criar projeto/credenciais.
5. **Decidir as Questões em aberto (§11)** — sobretudo fuso, auth do MVP e política de fricção — antes de codar o domínio.
6. **Começar pelo backbone do MVP:** `platform` (`couple_events` + eventbus + auth) → `identity` (pareamento) → `rhythm` (terços) → `activities` → `feedback`, com sync REST+tickle.

13. Referências

PEER-REVIEWED / PRIMÁRIAS (MAIS ROBUSTAS)

- Heyman, R. E., & Smith Slep, A. M. (2001). *The Hazards of Predicting Divorce Without Crossvalidation*. J. Family Psychology (PMC1622921).
- Levenson, R. W., & Gottman, J. M. (1985). *JPSP* 49:85-94 — excitação fisiológica prediz queda de satisfação.
- Carrère, S., & Gottman, J. M. (1999). *Family Process* — direção do soft startup.
- Kim, Capaldi & Crosby (2007). *J. Marriage & Family* — falha de replicação dos modelos afetivos.
- Baumeister, Bratslavsky, Finkenauer & Vohs (2001). *Bad Is Stronger Than Good*. Review of General Psychology.
- Doss et al. — OurRelationship RCT / IBCT (PMC5786273); versão com coach automatizado (ScienceDirect, 2023).
- BMC Psychology (2025), meta-análise de intervenções digitais para casais (s40359-025-03444-y).
- Park, Impett et al. — pairfam longitudinal, *exchange orientation* (~7.000 casais, 13 anos).
- Clark & Mills (1979; 1993); Clark, Mills & Powell — comunal vs. troca e "keeping track of needs".
- Walster (Hatfield), Walster & Berscheid (1978). *Equity: Theory and Research*.
- Deci (1971) / Lepper et al. — overjustification effect.
- Algoe (2012), *Find, Remind, Bind*; Algoe, Kurtz & Hilaire (2016, PMC4988174).
- Rogers, Howieson & Neame (2018). *I-language* (PMC5961625).
- Weinstein et al., *Who Enjoys Solitude?* (PMC9132342); estudo diádico de solitude (PMC10736816).
- Aron et al. / Reissman, Aron & Bergen (1993) — self-expansion. · SDT em mHealth (PMC8391751). · Koster et al. (2022, SAGE).
- Brown, Sokal & Friedman (2013) — retratação da "positivity ratio" de Losada (não confundir com o 5:1 de Gottman).

FRAMEWORKS / TEORIA CONSOLIDADA (ÚTEIS, SEM RCT ROBUSTO — ROTULAR COMO TAIS)

- Gottman Institute (Quatro Cavaleiros, 5:1, bids, Sound Relationship House, love maps) — **parte interessada**; os percentuais famosos (90%, 96%, 86/33) não resistem a escrutínio.
- Rosenberg, *Nonviolent Communication* (2003). · Stone & Heen, *Thanks for the Feedback* (2014) — modelo de praticantes, não RCT.
- Bowen Family Systems Theory (diferenciação do self). · Thibaut & Walker (1975) / Tyler / Leventhal (1980) — justiça procedural.
- Deci & Ryan — Self-Determination Theory. · Weiss (1980) — cunhagem de "sentiment override".

DIVULGAÇÃO / CRÍTICA (CONTEXTO, NÃO EVIDÊNCIA PRIMÁRIA)

- Laurie Abraham (*Slate*, 2010), "Can You Really Predict the Success of a Marriage in 15 Minutes?"; Andrew Gelman (crítica de *postdiction*).
- "Gamifying Intimacy" (Sage, 2025) — risco de comodificação afetiva; cases Paired / Lasting / Relish.

*Nota de calibração final: onde o app reivindicar embasamento científico, citar os **construtos e direções** (reduzir desprezo, aumentar apreço/responsividade, apoiar autonomia, normas comunitais) com linguagem de correlato ("associado a relações mais saudáveis"), evitando exibir os percentuais famosos — eles são o ponto mais atacável.*

Documento gerado a partir de pesquisa com verificação adversarial e design de arquitetura reconciliado · nome de trabalho "Terços" · 07/07/2026.